UNIVERSIDAD MODELO

Ingeniería en Desarrollo de Tecnología y Software

Reporte técnico

Juan Felipe Cervantes Alonzo 15233909
Kevin Emanuel Garabita Córdova 15233628
Eliam Matus Salvador 15233936
Emiliano Martínez Ramón 15233862
Victor Becerra Benavides 15234616
Jose Emilio Varguez Duran 15234637

10 de marzo del 2026

Introducción

El Internet de las Cosas (IoT) permite conectar dispositivos físicos a internet para recopilar, transmitir y analizar datos en tiempo real. En este proyecto se desarrolló un sistema de monitoreo ambiental distribuido utilizando múltiples nodos ESP32 capaces de medir variables ambientales como temperatura, humedad y luminosidad.

El sistema utiliza el protocolo MQTT para la comunicación entre dispositivos, un gateway que recibe y procesa los datos, y un dashboard web que permite visualizar la información en tiempo real y controlar actuadores de manera remota.

La arquitectura implementada está compuesta por cuatro componentes principales: nodos sensores basados en ESP32, un broker MQTT para la comunicación entre dispositivos, un backend desarrollado en Python con Flask que gestiona los datos recibidos y una interfaz web para visualizar la información y controlar el sistema.

Este proyecto permite aplicar conceptos clave de IoT como comunicación máquina a máquina, arquitecturas distribuidas, procesamiento de datos en tiempo real y control remoto de dispositivos.

Decisiones de diseño

Durante el desarrollo del sistema se tomaron diversas decisiones de diseño para garantizar una arquitectura funcional, modular y fácil de implementar.

Una de las decisiones principales fue utilizar el microcontrolador ESP32 como nodo sensor. El ESP32 fue seleccionado debido a que cuenta con conectividad WiFi integrada, múltiples pines de entrada y salida, bajo costo y gran compatibilidad con sensores y librerías del entorno Arduino. Esto facilita la implementación de nodos IoT sin necesidad de hardware adicional para conectividad de red.

Para la comunicación entre los nodos sensores y el servidor se eligió el protocolo MQTT. Este protocolo es ampliamente utilizado en aplicaciones IoT debido a su bajo consumo de ancho de banda, su modelo de comunicación basado en publicación y suscripción y su capacidad para manejar múltiples dispositivos conectados simultáneamente.

Como broker MQTT se utilizó Mosquitto, instalado en una Raspberry Pi que actúa como gateway del sistema. Mosquitto es una solución ligera, estable y fácil de configurar, lo que lo hace ideal para proyectos de IoT.

En el backend del sistema se implementó un subscriber en Python utilizando la librería paho-mqtt para recibir los datos publicados por los nodos ESP32. Estos datos se almacenan en un archivo JSON para mantener un registro histórico de las mediciones.

Posteriormente se desarrolló una API REST utilizando Flask, que permite acceder a los datos almacenados y enviar comandos de control a los nodos mediante MQTT.

Finalmente, se desarrolló un dashboard web interactivo utilizando las tecnologías web react con typescript tailwind. Este dashboard consume la API REST para mostrar los datos en tiempo real y permitir el control del relevador de cada nodo.

Esta arquitectura permite separar claramente las responsabilidades de cada componente del sistema, lo que facilita su mantenimiento, escalabilidad y futuras mejoras.

Retos técnicos

Durante el desarrollo del proyecto se presentaron diversos retos técnicos que requirieron investigación y ajustes en la implementación.

Uno de los principales retos fue lograr una comunicación estable entre los nodos ESP32 y el broker MQTT. En las primeras pruebas se presentaban desconexiones debido a problemas de red o pérdida de señal WiFi. Para solucionar este problema se implementaron mecanismos de reconexión automática tanto para la conexión WiFi como para la conexión al broker MQTT dentro del código del ESP32.

También se presentaron dificultades al integrar todos los componentes del sistema. Fue necesario asegurar que los topics MQTT utilizados por los nodos, el subscriber y la API coincidieran correctamente para garantizar que los mensajes se transmitieran y procesaran adecuadamente.

Adicionalmente, fue necesario implementar un mecanismo para almacenar los datos recibidos en formato JSON, permitiendo así que la API REST pudiera acceder a la información de manera rápida y sencilla.

Finalmente, la integración entre el dashboard web y la API requirió manejar correctamente las peticiones HTTP y la actualización automática de datos en tiempo real, lo cual se resolvió utilizando peticiones periódicas desde el frontend.

Análisis de datos

Durante la ejecución del sistema se recopilaron múltiples lecturas de temperatura, humedad y luminosidad provenientes de los nodos ESP32.

Los datos recopilados permitieron observar tendencias en el comportamiento de las variables ambientales monitoreadas. Por ejemplo, la temperatura presentó variaciones a lo largo del tiempo dependiendo de las condiciones del entorno donde se encontraban los sensores.

La humedad también mostró cambios graduales que reflejan las condiciones ambientales del lugar de prueba.

En cuanto a la luminosidad, los valores obtenidos del sensor LDR permitieron observar diferencias claras entre condiciones de mayor y menor iluminación.

Mediante el dashboard web se generaron gráficas que muestran la evolución de estas variables en el tiempo. Estas gráficas permiten analizar el comportamiento de cada nodo y comparar las mediciones entre diferentes ubicaciones.

En general, los resultados obtenidos demuestran que el sistema es capaz de recopilar y visualizar datos ambientales en tiempo real de manera confiable.

Conclusiones

El desarrollo de este proyecto permitió implementar un sistema completo de monitoreo ambiental basado en tecnologías IoT. Se logró integrar hardware, software y comunicación en red para construir una solución funcional capaz de recopilar datos ambientales y visualizarlos en tiempo real.

El uso del protocolo MQTT permitió establecer una comunicación eficiente entre los nodos sensores y el gateway del sistema. Asimismo, la utilización de una API REST facilitó la integración con el dashboard web, permitiendo acceder a los datos de manera estructurada.

El sistema desarrollado demuestra cómo es posible construir soluciones IoT utilizando tecnologías accesibles y de bajo costo, manteniendo una arquitectura modular que puede escalar fácilmente para incluir más sensores o funcionalidades.

Mejoras futuras

Aunque el sistema desarrollado cumple con los objetivos del proyecto, existen varias mejoras que podrían implementarse en futuras versiones.

Una de las mejoras más importantes sería reemplazar el almacenamiento en archivos JSON por una base de datos como MongoDB o PostgreSQL, lo que permitiría manejar grandes volúmenes de datos de manera más eficiente.

Otra posible mejora sería implementar un sistema de autenticación y seguridad en la API para evitar accesos no autorizados al sistema.

También se podría mejorar el dashboard web incorporando más herramientas de análisis de datos, como filtros de fechas, comparación avanzada entre nodos y exportación de datos en diferentes formatos.

Además, el sistema podría ampliarse integrando más tipos de sensores, como sensores de calidad del aire o sensores de presión atmosférica.

Finalmente, una mejora importante sería desplegar el sistema en la nube, permitiendo acceder al dashboard desde cualquier ubicación y facilitando el monitoreo remoto del sistema.